

Instruction Register

Test Report

Comprehensive Verification of PolyFlop8 Instruction Register

Document ID:	TC-IR-001
Revision:	1.0
Date:	January 5, 2026
Test Level:	Unit/Block Level
Test Engineer:	Artem Horiunov
Status:	PASS

1 TEST OVERVIEW

1.1 Unit Under Test (UUT)

Component	Instruction Register for PolyFlop8 Processor
VHDL Entity	InstructionReg
Source File	InstructionReg.vhd, tb_InstructionReg.vhd
Test Environment	Vivado v2025.1 (64-bit) SW Build: 6140274 on Thu May 22 00:12:29 MDT 2025 IP Build: 6138677 on Thu May 22 03:10:11 MDT 2025 SharedData Build: 6139179 on Tue May 20 17:58:58 MDT 2025
Clock Frequency	100MHz (10ns period)
Data Width	16-bit

1.2 Scope

This test report documents the verification of the Instruction Register for the PolyFlop8 Processor. The Instruction Register is responsible for storing the current instruction fetched from memory and providing it to the decoder and control unit.

1.3 Test Objectives

- Verify correct operation of the Instruction Register during reset
- Validate data write and hold functionality
- Confirm asynchronous reset behavior
- Ensure 100% code coverage of Instruction Register behavioral description

2 REQUIREMENTS COVERAGE

Req ID	Requirement Description	Test Case ID	Status
TR-IR-01	Verify data write functionality	TC-IR-001-02	PASS
TR-IR-02	Verify data hold functionality	TC-IR-001-03	PASS
TR-IR-03	Verify asynchronous reset functionality	TC-IR-001-01, TC-IR-001-04	PASS
TR-IR-04	Verify stability during asynchronous re-set	TC-IR-001-04	PASS
TR-IR-05	Verify data integrity during hold	TC-IR-001-03	PASS
TR-IR-06	Verify data update on clock edge	TC-IR-001-02	PASS

Table 1: Requirements Coverage Matrix

3 DETAILED TEST CASES

3.1 Test Case TC-IR-001-01: Reset and Initial State

Requirement: TR-IR-03

Purpose: Verify asynchronous reset functionality and initial state.

Status: PASS

3.2 Test Case TC-IR-001-02: Data Write

Requirement: TR-IR-01, TR-IR-06

Purpose: Verify data write functionality on clock edge.

Status: PASS

3.3 Test Case TC-IR-001-03: Data Hold

Requirement: TR-IR-02, TR-IR-05

Purpose: Verify data hold functionality when write is disabled.

Status: PASS

3.4 Test Case TC-IR-001-04: Asynchronous Reset

Requirement: TR-IR-03, TR-IR-04

Purpose: Verify asynchronous reset behavior during operation.

Status: PASS

4 SOURCE CODES

4.1 InstructionReg.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity InstructionReg is
    Port (
        clk      : in  STD_LOGIC;           -- System clock
        rst      : in  STD_LOGIC;           -- Asynchronous reset
        ir_en    : in  STD_LOGIC;           -- Write enable (Active
        --      ↗ High)
        -- Data input (usually from Memory/ROM output)
        data_in   : in  STD_LOGIC_VECTOR (15 downto 0);
        -- Output to Decoder and Control Unit
        ir_data_out : out STD_LOGIC_VECTOR (15 downto 0)
    );
end InstructionReg;

architecture Behavioral of InstructionReg is
    -- Internal register storing the current instruction
    signal instruction_reg : std_logic_vector(15 downto 0) := (others => '0');
begin
    process(clk, rst)
    begin
        -- 1. Asynchronous reset
        if rst = '1' then
            instruction_reg <= (others => '0'); -- Reset to NOP (all zeros)
        end if;
    end process;
end Behavioral;
```

```

-- 2. Synchronous write
elsif rising_edge(clk) then
    -- Register latches new instruction ONLY if Control Unit allows (
    --   ↳ FETCH state)
    if ir_en = '1' then
        instruction_reg <= data_in; -- Latch data from input
    end if;
    -- If ir_en = '0', instruction remains latched (stable for DECODE/
    --   ↳ EXECUTE)
end if;
end process;

-- Output the signal externally
ir_data_out <= instruction_reg;

end Behavioral;

```

4.2 tb_InstructionReg.vhd

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

-- Testbench entity (empty, as it's a simulation)
entity tb_InstructionReg is
end tb_InstructionReg;

architecture Behavioral of tb_InstructionReg is

    -- 1. Component Declaration (Consistent with your code)
    component InstructionReg is
        Port (
            clk          : in  STD_LOGIC;
            rst          : in  STD_LOGIC;
            ir_en        : in  STD_LOGIC;
            data_in       : in  STD_LOGIC_VECTOR (15 downto 0);
            ir_data_out   : out STD_LOGIC_VECTOR (15 downto 0)
        );
    end component;

    -- 2. Internal signals for connecting to UUT (Unit Under Test)
    signal clk_tb        : STD_LOGIC := '0';
    signal rst_tb        : STD_LOGIC := '0';
    signal ir_en_tb      : STD_LOGIC := '0';
    signal data_in_tb    : STD_LOGIC_VECTOR (15 downto 0) := (others => '0');
    signal ir_data_out_tb : STD_LOGIC_VECTOR (15 downto 0);

    -- Clock period constant (e.g., 10 ns -> 100 MHz)
    constant CLK_PERIOD : time := 10 ns;

begin

    -- 3. Instance of the Unit Under Test
    UUT: InstructionReg
    port map (
        clk          => clk_tb,
        rst          => rst_tb,
        ir_en        => ir_en_tb,
        data_in       => data_in_tb,
        ir_data_out   => ir_data_out_tb
    );

```

```

-- 4. Clock generation process
clk_process : process
begin
    clk_tb <= '0';
    wait for CLK_PERIOD/2;
    clk_tb <= '1';
    wait for CLK_PERIOD/2;
end process;

-- 5. Stimulus process (Main test scenario)
stim_proc: process
begin
    -- Initialization
    report "Rozpoczecie_testu_Instruction_Register_(TC-IR-001)";
    rst_tb <= '1';      -- Active reset at start
    ir_en_tb <= '0';
    data_in_tb <= x"0000";
    wait for 100 ns;    -- Time to stabilize after start

    -----
    -- TEST CASE 1: Reset Verification (TR-IR-03)
    -----
    -- Check if output is 0x0000 after reset
    assert ir_data_out_tb = x"0000"
        report "Blad_TC1:_Reset_nie_wyzerowal_rejestru" severity error;

    -- Release reset
    rst_tb <= '0';
    wait for CLK_PERIOD;

    -----
    -- TEST CASE 2: Data Write - Enabled (TR-IR-01, TR-IR-04, TR-IR-06)
    -----
    report "TC2:_Test_zapisu_danych_(ir_en=_1)";

    -- Set data and enable write
    ir_en_tb <= '1';
    data_in_tb <= x"AAAA"; -- Pattern 1010...
    wait for CLK_PERIOD;   -- Wait for clock edge

    -- Check if data was latched
    assert ir_data_out_tb = x"AAAA"
        report "Blad_TC2a:_Dane_0xAAAA_nie_zostaly_zapisane" severity error;

    -- Change data to another pattern (Pattern 0101...)
    data_in_tb <= x"5555";
    wait for CLK_PERIOD;

    assert ir_data_out_tb = x"5555"
        report "Blad_TC2b:_Dane_0x5555_nie_zostaly_zapisane" severity error;

    -----
    -- TEST CASE 3: Data Hold - Disabled (TR-IR-02, TR-IR-05)
    -----
    report "TC3:_Test_podtrzymania_danych_(ir_en=_0)";

    -- Disable write, but change input data
    ir_en_tb <= '0';
    data_in_tb <= x"FFFF"; -- Attempt to write all ones
    wait for CLK_PERIOD * 2; -- Wait two cycles

    -- Output should still hold the old value (0x5555), not the new one (0
    ↪ xFFFF)

```

```

assert ir_data_out_tb = x"5555"
    report "Blad_TC3: Rejestr nadpisał dane mimo ir_en='0'" severity
        ↪ error;

-----
-- TEST CASE 4: Asynchronous Reset (Additional stability test)
-----
report "TC4: Test asynchronicznego resetu w trakcie pracy";

ir_en_tb <= '1';          -- Write enabled
data_in_tb <= x"BEEF";    -- Data at input
wait for CLK_PERIOD;      -- Latch 0xBEEF

assert ir_data_out_tb = x"BEEF" report "Blad_TC4a: Przygotowanie danych
    ↪ nieudane" severity failure;

rst_tb <= '1';            -- Sudden reset
wait for 2 ns;            -- Short delay (less than clock cycle)

-- Should reset immediately, without waiting for clock edge
assert ir_data_out_tb = x"0000"
    report "Blad_TC4b: Reset asynchroniczny nie zadzialal natychmiast"
        ↪ severity error;

rst_tb <= '0';
ir_en_tb <= '0';

-----
-- End
-----
report "Test Instruction Register zakonczony pomyslnie.";
wait; -- Stop simulation
end process;

end Behavioral;

```

5 SIMULATION LOG

```

Time resolution is 1 ps
Note: Rozpoczecie testu Instruction Register (TC-IR-001)
Time: 0 ps Iteration: 0 Process: /tb_InstructionReg/stim_proc File: F:/
    ↪ Projects/PolyFlop8-MCU/Polyflop8/Polyflop8.srscs/sources_1/new/
    ↪ Unit_test_bench/InstructionRegister_TB.vhd
Note: TC2: Test zapisu danych (ir_en = '1')
Time: 110 ns Iteration: 0 Process: /tb_InstructionReg/stim_proc File: F:/
    ↪ Projects/PolyFlop8-MCU/Polyflop8/Polyflop8.srscs/sources_1/new/
    ↪ Unit_test_bench/InstructionRegister_TB.vhd
Note: TC3: Test podtrzymania danych (ir_en = '0')
Time: 130 ns Iteration: 0 Process: /tb_InstructionReg/stim_proc File: F:/
    ↪ Projects/PolyFlop8-MCU/Polyflop8/Polyflop8.srscs/sources_1/new/
    ↪ Unit_test_bench/InstructionRegister_TB.vhd
Note: TC4: Test asynchronicznego resetu w trakcie pracy
Time: 150 ns Iteration: 0 Process: /tb_InstructionReg/stim_proc File: F:/
    ↪ Projects/PolyFlop8-MCU/Polyflop8/Polyflop8.srscs/sources_1/new/
    ↪ Unit_test_bench/InstructionRegister_TB.vhd
Note: Test Instruction Register zakonczony pomyslnie.
Time: 162 ns Iteration: 0 Process: /tb_InstructionReg/stim_proc File: F:/
    ↪ Projects/PolyFlop8-MCU/Polyflop8/Polyflop8.srscs/sources_1/new/
    ↪ Unit_test_bench/InstructionRegister_TB.vhd

```

6 PASS/FAIL CRITERIA

6.1 Success Criteria

Criteria	Description
Functional	All test cases execute without assertion failures
Timing	No timing violations (Instruction Register is synchronous)
Coverage	100% statement coverage, 100% branch coverage achieved
Consistency	Identical results across multiple simulation runs
Documentation	All test results properly documented and signed off

6.2 Failure Conditions

- Any single assertion failure during simulation
- Code coverage less than 100% for Instruction Register behavioral code
- Timing violations or simulation warnings
- Inconsistent results between simulation runs
- Missing or incomplete test documentation

6.3 Coverage Goals

- **Statement Coverage:** 100% of Instruction Register process statements executed
- **Branch Coverage:** 100% of case statement branches taken
- **Condition Coverage:** All write and reset conditions tested
- **Toggle Coverage:** All signal bits toggled 0→1 and 1→0
- **FSM Coverage:** N/A (Instruction Register is not a state machine)

7 SPECIAL TEST CONSIDERATIONS

7.1 Edge Cases

- Test asynchronous reset during data write
- Verify data integrity during hold
- Test boundary conditions for data input
- Verify correct behavior during clock edge transitions
- Test all possible multiplexer combinations

7.2 Timing Considerations

- Instruction Register is synchronous for write and asynchronous for reset
- Monitor for glitches or hazards during input transitions
- Verify stable outputs within specified propagation delay
- Test with minimum, typical, and maximum delay models

7.3 Integration Considerations

- Verify interface compatibility with Memory and Control Unit
- Test handshake signals with Control Unit
- Validate output format matches processor expectations
- Confirm timing compatibility with other processor components